

Version 2.0

January 2026



OBJECT-ORIENTED PROGRAMMING

MODULE 3 -INHERITANCE, CONSTRUCTION, OVERRIDING

Author:

Ir. Galih Wasis Wicaksono, S.Kom., M.Cs.

Yossua Agung Budianto

Muhammad Budi Kusuma

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE TARGETS.....	3
PREPARATION.....	3
KEYWORDS.....	3
1. Inheritance.....	4
1.1 Why Should I Understand This?.....	4
1.2 Formal Definition.....	5
1.3 Concept Elaboration.....	5
1.3.1 Philosophy Generalization and Specialization.....	5
1.3.2 Mechanism Memory in Object Instance Inheritance.....	6
1.3.3 Single Inheritance and Class Hierarchy.....	6
1.3.4 Work With Inheritance.....	6
2. Constructor.....	8
2.1 Why Should I Understand This.....	8
2.2 Formal Definition.....	8
2.3 Concept Elaboration.....	9
2.3.1 Anatomy of a Constructor.....	9
2.3.2 Constructor Chaining.....	9
2.3.3 Role of Implicit and Explicit Super.....	9
2.3.3 Super as Reference to Members (Shadowing).....	9
2.3.4 Practice with Constructor.....	10
3. Overriding and Overloading.....	13
3.1 Why Should I Understand This?.....	13
3.2 Formal Definition.....	13
3.3 Concept Elaboration.....	13
3.3.1 Overriding and Dynamic Binding (Runtime).....	13
3.3.2 Overload and Static Binding (Compile-time).....	14
3.3.3 Work with Overloading and Overriding.....	14
Exercise.....	17
Assignment.....	19
Assessment.....	22
Percentage Assessment.....	22
Closing Statement?.....	23



INTRODUCTION

OBJECTIVES

1. To enable students to understand the fundamental principles of inheritance
2. To provide students with a clear understanding of constructor
3. To equip students with knowledge of the difference between overloading and overriding

MODULE TARGETS

1. Students are able to implement inheritance in Java programs
2. Students are able to design and implement constructors with proper constructor chaining

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللَّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop or personal computer
3. IntelliJ IDEA
4. "FULLL POWEEEEERRRRR"

KEYWORDS

Constructor, Super, Inheritance, Overloading, Overriding



1. Inheritance

1.1 Why Should I Understand This?

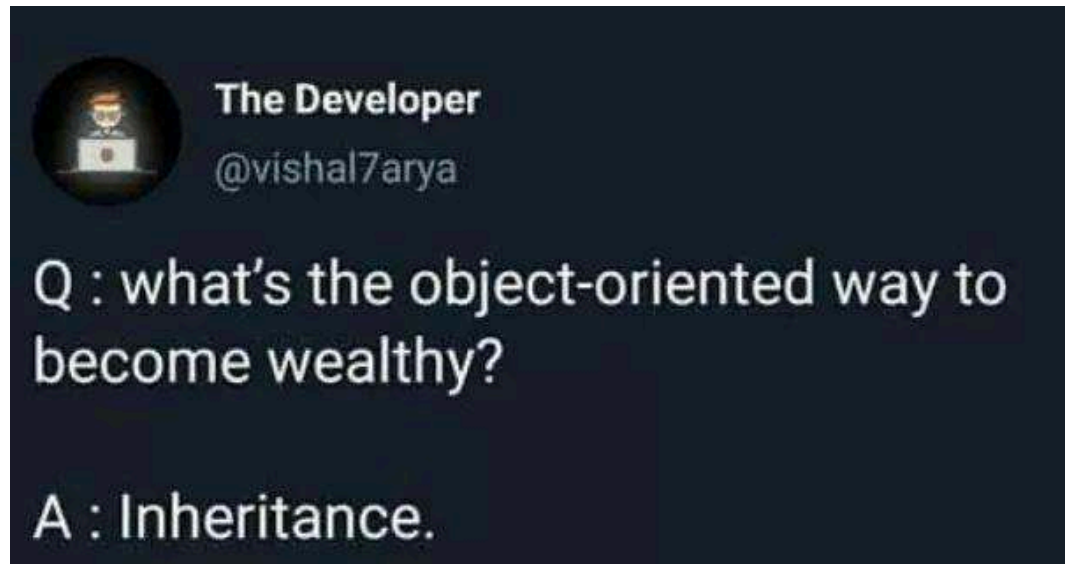


Understanding inheritance is key to writing efficient code. Imagine if you had to design a car, truck, and motorcycle from scratch, when all three have the same basic components like tires, an engine, and fuel. Without this concept, you would waste time rewriting the same code again and again for each object.

You'll also learn how to create a logical hierarchy, where it's enough to define general features in the parent class and let the child classes inherit those features. This makes your code more efficient, maintainable, and applicable to real-world cases.



1.2 Formal Definition



Inheritance is a fundamental mechanism in OOP where a new class (subclass or child class) adopts state (attributes) and behavior (methods) from an existing class (superclass or parent class). Object instance inheritance specifically refers to the memory layout during runtime, where instance objects from subclasses are physically structured to contain the data structure from the superclass.

Inheritance is a programming principle where one class has the ability to inherit properties and methods from another class. The concept of inheritance utilizes "code reuse" to avoid duplicating code in programs.

1.3 Concept Elaboration

1.3.1 Philosophy Generalization and Specialization

Inheritance works based on the principle of hierarchy from generalization to specialization. In system design, we usually group objects by their similarities.

Generalization is the process to take similarity from some class and make it a superclass. On the other hand, Specialization is a process to make subclasses more detailed. This relation is known as the "Is-A" Relation.

For example, imagine hierarchy is "Vehicle". Vehicle is a form of generalization that always tires and machine. "Sport cars" is a form of specialization from vehicles. It inherits tires and machines, but adds special features like turbo boosters. In code, this means the subclass won't need to redefine what it has from the superclass, but only focus on adding unique features for itself.



1.3.2 Mechanism Memory in Object Instance Inheritance

This part is often missed by most people but is very crucial: what happens in the computer's RAM when an inheritance object is created? When you instantiate a subclass (e.g., `new SportsCar()`), the operating system allocates one block of memory in the Heap. Interestingly, this memory block is not only filled with `SportsCar` data, but also contains memory slots for the variables of the `Vehicle` class (superclass).

Physically in memory, a child object is a "wrapper" that wraps the data from its parent. This is called object instance inheritance. Even though parent data (e.g., `private int speed`) is saved in the child object, access to that data is still limited by access modifiers. So, the child object carries the "DNA" of its parent within it, making them a unified entity while the program is running.

1.3.3 Single Inheritance and Class Hierarchy

In many modern programming languages like Java, the rule of single inheritance is implemented. This means a child class can only have one parent class explicitly. This rule is designed to avoid logical ambiguity known as the "Diamond Problem," where a situation becomes confusing if a child inherits two methods with the same name from two different parents.

As a result of this rule, the inheritance structure will always form a tree structure, where one root (usually the `Object` class) branches downward. If you need behavior from multiple sources, the solution is not multiple class inheritance, but using the interface mechanism.

1.3.4 Work With Inheritance



We've already discussed the theory of inheritance. Now, how do we work with inheritance in our code? We use the keyword `extends`. How do we use it? Check this out. Let's say we have a class named `Animal` like this:

```
public class Animal { 1 usage 1 inheritor
    String nama; 1 usage
    int legs; 1 usage

    public void berkembangBiak(){ 1 usage 1 override
        System.out.println("Hewan berkembang biak");
    }
}
```

This is called the parent class, and we have a child class called `Mamalia`:

```
public class Mamalia extends Animal{ 2 usages
    String jenisBulu; 1 usage

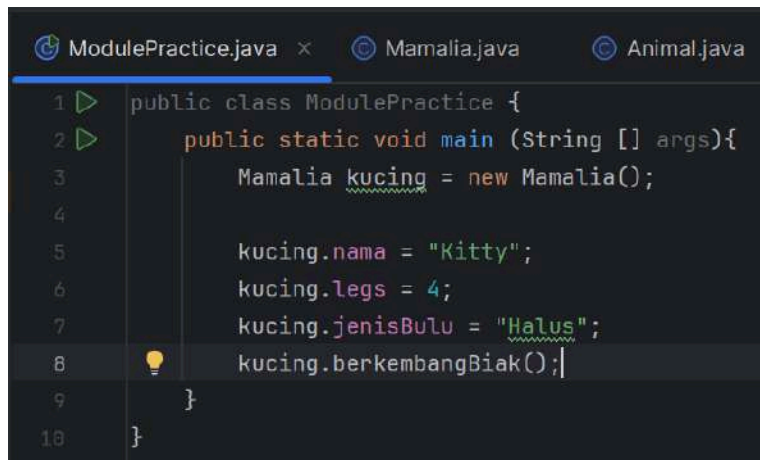
    public void berkembangBiak(){ 1 usage
        System.out.println("melahirkan anak");
    }
}
```

As you can see, we created a child class and added the word `extends` after the name of our class, followed by the parent class. After we create the class, what can we do? Let's try creating objects with our child class:

```
public class ModulePractice {
    public static void main (String [] args){
        Mamalia kucing = new Mamalia();
    }
}
```



The object from the child class can access attributes and methods:



```

1 public class ModulePractice {
2     public static void main (String [] args){
3         Mamalia kucing = new Mamalia();
4
5         kucing.nama = "Kitty";
6         kucing.legs = 4;
7         kucing.jenisBulu = "Halus";
8         kucing.berkembangBiak();
9     }
10 }

```

As we can see, we can access the attributes and methods from the parent class even though the object is the child.

2. Constructor

2.1 Why Should I Understand This

Every object that is created in the system must be in a "healthy" or valid state. The constructor is a mechanism that ensures this. Understanding the interaction between constructors and the `super` keyword is very important to ensure that object instances follow the correct hierarchy order. Misunderstanding this can create bugs where a child object tries to use data that was never initialized.

2.2 Formal Definition



A constructor is a special block of code that executes automatically when an instance object is created. `super` is a reference variable used in a subclass to refer to members (constructor, method, or variable) directly from the superclass.

A constructor is a special method that initializes values when an object is created. When we create an object, the constructor runs first, even if we don't explicitly call it.

2.3 Concept Elaboration

2.3.1 Anatomy of a Constructor

To create a constructor, we need to declare it first, and it's very simple. The syntax is: `[Name of class]() { Body };`. If we want to use a constructor with parameters: `[Name of class](parameter) { Body };`.

2.3.2 Constructor Chaining

In the concept of inheritance, the process of creating an object triggers a domino effect called constructor chaining. For example: a child can't exist if the parent doesn't exist. Because of this, when a child class constructor is called, the first thing it does, even before initializing its own variables, is call the parent's constructor.

2.3.3 Role of Implicit and Explicit Super

Programming languages usually help programmers by using `super()` implicitly. If you don't write anything in the child constructor, the compiler will assume you're calling the parent constructor without parameters.

However, the situation gets complicated if the parent only has constructors that require parameters (e.g., `Parent(name)`). In this case, the automatic help from the compiler stops working. You as a programmer must explicitly write `super(value)` as the first line of the child's constructor. The golden rule is: calling `super()` should be the first line in the body of a constructor. However, in JDK 25, we can write calculations before using `super()`.

2.3.3 Super as Reference to Members (Shadowing)

Besides constructors, `super` is used to solve problems called variable shadowing or method overriding. Sometimes, a child has a variable or method with the same name as its parent. If we write `huntPrey()` in the child class, the system will refer to the child's version..

To access the parent's version that is being "shadowed," we need to use the `super` prefix. For example, `super.calculateSalary()` will logically call the "salary" method from the parent, while `this.calculateSalary()` will logically call the "salary" method from the child. This technique is usually used when we want to extend functionality, not completely replace it. We call the old function with `super`, then add new code below.



2.3.4 Practice with Constructor

Let's practice creating a constructor with the code we learned before. As you saw earlier, we set our attributes one by one, which isn't very readable. With a constructor, it'll be more efficient and, of course, more readable. Let's try with the Animal class. We're going to add a constructor to it:

```
ModulePractice.java  Mamalia.java  Animal.java x
1 public class Animal { 1 usage 1 Inheritor
2     String nama; 1 usage
3     int legs; 1 usage
4
5     public Animal () { 1 usage
6         System.out.println("Ini adalah hewan");
7     }
8
9     public void berkembangBiak() { 1 usage 1 override
10        System.out.println("Hewan berkembang biak");
11    }
12 }
```

If we create an object from the Animal class, the output will look like this:

```
ModulePractice.java  Mamalia.java  Animal.java
1 public class ModulePractice {
2     public static void main (String [] args){
3         Animal hewan = new Animal();
4     }
5 }
```

```
Ini adalah hewan
Process finished with exit code 0
```



It's that simple. If we want to add parameters, the code will look like this:

```
ModulePractice.java Mamalia.java Animal.java x
1 public class Animal { 3 usages 1 inheritor 2 related problems
2     String nama; 1 usage
3     int legs; 1 usage
4
5     public Animal (String nama, int legs){ 1 usage 2 related problems
6         this.nama = nama;
7         this.legs = legs;
8     }
9
10    public void berkembangBiak(){ no usages 1 override
11        System.out.println("Hewan berkembang biak");
12    }
13 }
```

We should use the word `this` to refer to the class itself (class attributes), so it doesn't get mixed up with the parameters. As you can see, there's an error because we created the object before without parameters. Let's fix it:

```
ModulePractice.java x Mamalia.java Animal.java
1 public class ModulePractice {
2     public static void main (String [] args){
3         Animal hewan = new Animal( nama: "Kucing", legs: 4);
4     }
5 }
```

We've just added parameters to our object, and it's simpler than before. We have one more problem, the Mamalia class has an error. Why?



As we discussed before about the role of implicit and explicit super, the analogy is like an ID card. Imagine working with an ID card system. The government says everyone



automatically has an empty ID card. If the parent (government) has a stock of empty ID cards (default constructor), then for every child class, the system automatically takes one empty ID card for it. You don't have to request it, the system already knows (this is an example of implicit).

Then the government changes the rules. Now every ID card must have a full name on it. The system will be confused when you have a child (class), the system should create the parent first, but it needs the name of the parent. The system doesn't know the name, so the system throws an error. Because the system is confused, you as a programmer must give clear instructions like "Hey system, create a child class and its parent, and give it the name 'Budi'."

The code will look like this:



```

1 public class Mamalia extends Animal{ no usages
2     String jenisBulu; 1 usage
3
4     public Mamalia (String nama, int legs, String jenisBulu){ no usages
5         super(nama, legs);
6         this.jenisBulu = jenisBulu;
7         System.out.println("Kelas Mamalia dibuat");
8     }
9
10    public void berkembangBiak(){ no usages
11        System.out.println("melahirkan anak");
12    }
13 }

```

The `super` will receive its own parameters to provide the name and legs from the parent class (Animal), then we add parameters for the child's own attributes.

If we create the object and run it, it should look like this:



```

1 public class ModulePractice {
2     public static void main (String [] args){
3         Mamalia mamalia = new Mamalia( nama: "Kucing", legs: 4, jenisBulu: "Bulu Pendek");
4     }
5 }

```

```
Kelas Mamalia dibuat
```

```
Process finished with exit code 0
```



3. Overriding and Overloading

3.1 Why Should I Understand This?

The real world is full of context. The word "open" can mean open a door, open an app, or even open up a conversation. In programming, the ability to use one name for many behaviors is called polymorphism. Understanding overriding and overloading enables you to write flexible, readable, and intuitive code without creating hundreds or thousands of function names that do similar things.

3.2 Formal Definition



Overriding is a subclass's ability to provide a specific reimplement of a method that is already provided by the superclass. **Overloading** is the ability to create methods with the same name in the same class but with different parameters.

3.3 Concept Elaboration

3.3.1 Overriding and Dynamic Binding (Runtime)

Overriding is at the core of dynamic polymorphism. This happens when a child class feels that the implementation of a method from the parent class is no longer relevant or needs to be more specific. The child then re-declares the method with the same name and the same parameters.

The magic of overriding lies in the mechanism of dynamic binding. Which method will be called is not determined when the code is compiled, but when the program is running (runtime). The Virtual Machine will check what object it actually holds. If a variable of type `Animal` actually contains a `Cat` object, then the `sound()` method from `Cat` will be called. This makes it possible to write code that is very general but behaves specifically depending on the object being processed..



3.3.2 Overload and Static Binding (Compile-time)

Unlike overriding, overloading is static polymorphism that happens at compile time. The goal is user convenience. The compiler creates different methods with the same name depending on the parameter "fingerprint."

For example, `print(int number)` and `print(String text)` are two completely different methods to the compiler, even though they have the same name. The compiler already decides which method will be called before the program runs (static binding). The absolute rule of overloading is that the parameter list must be different (either the number of parameters or the data types). Changing only the return value is not enough for overloading and will cause a compiler error.

3.3.3 Work with Overloading and Overriding

Let's try using overloading and overriding in our program. Let's continue from the previous program:

```
ModulePractice.java  Mamalia.java  Animal.java x
1 public class Animal { 3 usages 1 inheritor
2     String nama; 1 usage
3     int legs; 1 usage
4
5     public Animal (String nama, int legs){ 1 usage
6         this.nama = nama;
7         this.legs = legs;
8     }
9
10    public void berkembangBiak(){ no usages 1 override
11        System.out.println("Hewan berkembang biak");
12    }
13 }
```

As you can see, we have a method named `reproduce()` from the `Animal` class (parent), and if we want to use it in the `Mamalia` class, we have to use overriding:

```
ModulePractice.java  Mamalia.java x  Animal.java
1 public class Mamalia extends Animal{ 2 usages
2     String jenisBulu; 1 usage
3
4     public Mamalia (String nama, int legs, String jenisBulu){ 1 usage
5         super(nama, legs);
6         this.jenisBulu = jenisBulu;
7         System.out.println("Kelas Mamalia dibuat");
8     }
9
10    @Override no usages
11    public void berkembangBiak(){
12        System.out.println("mamalia melahirkan anak");
13    }
14 }
```

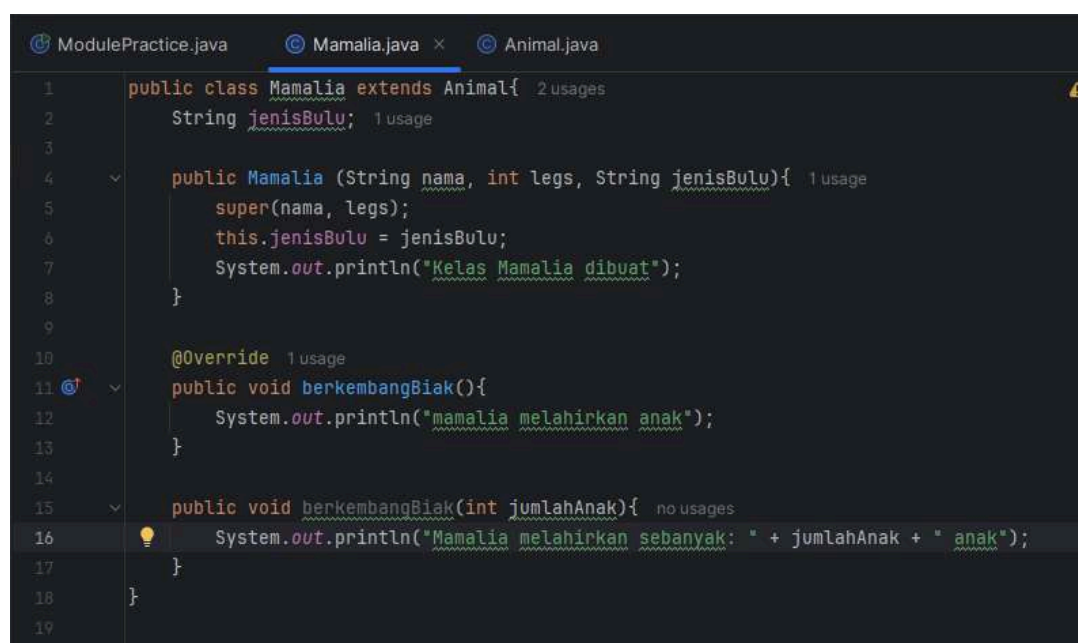


Using the `@Override` annotation, we can use the same method with the same name but different execution. If we run it, it'll look like this:

```
Kelas Mamalia dibuat
mamalia melahirkan anak

Process finished with exit code 0
```

What if we want to add a parameter to the child class? That's called overloading. We'll create the same method in the same class and add or remove parameters. This is important to remember:



```
ModulePractice.java  Mamalia.java  Animal.java

1  public class Mamalia extends Animal{ 2 usages
2      String jenisBulu; 1 usage
3
4      public Mamalia (String nama, int legs, String jenisBulu){ 1 usage
5          super(nama, legs);
6          this.jenisBulu = jenisBulu;
7          System.out.println("Kelas Mamalia dibuat");
8      }
9
10     @Override 1 usage
11     public void berkembangBiak(){
12         System.out.println("mamalia melahirkan anak");
13     }
14
15     public void berkembangBiak(int jumlahAnak){ no usages
16         System.out.println("Mamalia melahirkan sebanyak: " + jumlahAnak + " anak");
17     }
18 }
19
```

As we can see, we have 2 methods with the same name but different parameters. If we call the method with a parameter, it looks like this:



```
ModulePractice.java  Mamalia.java  Animal.java

1  public class ModulePractice {
2      public static void main (String [] args){
3          Mamalia mamalia = new Mamalia( nama: "Kucing", legs: 4, jenisBulu: "Bulu Pendek");
4
5          mamalia.berkembangBiak();
6
7          mamalia.berkembangBiak( jumlahAnak: 3);
8      }
9  }
```



The output will look like this:

```
Kelas Mamalia dibuat  
mamalia melahirkan anak  
Mamalia melahirkan sebanyak: 3 anak  
  
Process finished with exit code 0
```

Overloading can also be used on constructors, but for this one, you have to try it yourself.



Exercise

Continue from the previous work:

We've added a new class named `Character.java`, and now this is our new parent class. Now add new classes for hero types including `Archer`, `Mage`, and `Warrior`. Every Hero class has its own special method but shares the same attributes from `Character`. Here are the details:

1. Add a constructor for all attributes in the `Character` class

```
Warrior warior = new Warrior(nameChar, ageChar, gender, weapon, skin);

System.out.println("\n===== CHARACTER CREATED =====");
warior.displayCharInfo();

warior.attack();
```

2. Add a new method in `Character` named `attack()`
3. Every child class (`Archer`, `Mage`, `Warrior`) must override `attack()`

```
Archer archer = new Archer(nameChar, ageChar, gender, arrowCount, bowType);

System.out.println("\n===== CHARACTER CREATED =====");
archer.displayCharInfo();

archer.attack();
```

```
Mage mage = new Mage(nameChar, ageChar, gender, element, mana);

System.out.println("\n===== CHARACTER CREATED =====");
mage.displayCharInfo();
```

4. Mage must have a method `castUltimateSpell()`
5. Archer must use overloading on `attack()` with a parameter `numberOfArrows`
6. Warrior must also use overloading with a parameter `specialMove`
7. Here's the code for the `Warrior` class:



```

1 public class Warrior extends Character{
2     String weapon;
3     String skin;
4
5     public Warrior (String name, int age, boolean gender, String weapon, String skin) {
6         System.out.println("Warrior Created!");
7         super(name, age, gender);
8         this.weapon = weapon;
9         this.skin = skin;
10    }
11
12    @Override
13    public void displayCharInfo() {
14        super.displayCharInfo();
15        System.out.println("Class: Warrior");
16        System.out.println("Weapon: " + weapon);
17        System.out.println("Skin: " + skin);
18    }
19
20    @Override
21    public void attack() {
22        System.out.println(name + " attacks with a " + weapon + "!");
23    }
24
25    public void attack(String specialMove) {
26        System.out.println(name + " performs a special move: " + specialMove + " with a "
27        + weapon + "!");
28    }
29

```

Here's what you should understand:

1. What keyword do we use to inherit from a parent class to a child class?
2. What does the `super` keyword do?
3. Based on JDK 25, can we declare the `super` keyword not at the top of the constructor?
4. What's the difference between overriding and overloading?
5. What keyword do we use when we want to use overriding?



Assignment

The mayor is now expanding the data for infrastructure. Not only basic buildings—now the system must be able to differentiate between buildings like hospitals and so on. The Building class is now considered a parent class. Here's what you need to do:

1. Create a new class called Hospital and add 2 more classes with different names (Cinema, Apartment, etc.) that must be commercial buildings. Here's the code for the Hospital class:

```
public class Hospital extends Building {
    int numberOfBeds;

    //Todo: Constructor here

    //TODO: Override showbuilding method here

}
```

2. Since you're adding commercial buildings, these buildings have monthly revenue. Make every commercial class have a `monthlyRevenue` variable
3. Hospital and the other classes must override the `showBuilding()` method and create a new building method
4. Add new class variables for initial data and call the `showBuilding()` method for each object
5. Make every attribute private
6. Here's the code for Building and Main:

Building.Java

```
import java.util.Scanner;

public class Building {
    private String name;
    private String address;
    private int numberOfFloors;

    public Building (String name, String address, int numberOfFloors) {
        this.name = name;
        this.address = address;
        this.numberOfFloors = numberOfFloors;
    }

    public void showbuildings() {
        System.out.println();
    }
}
```



```

        System.out.println("=====");
        System.out.println("Building Name: " + name);
        System.out.println("Building Address: " + address);
        System.out.println("Number of Floors: " + numberOfFloors);
        System.out.println("=====");
    }
}

```

Main.Java

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int option;
        do {
            System.out.println("Welcome to Smart City Management System");

            System.out.println("1. Add New Building");
            System.out.println("2. Add New Hospital");
            System.out.println("3. View All Buildings");
            System.out.println("4. Exit");
            System.out.printf("Please select an option: ");
            option = scanner.nextInt();
            scanner.nextLine();

            switch (option) {
                case 1:
                    System.out.println("Enter Building Name: ");
                    String name = scanner.nextLine();
                    System.out.println("Enter Building Address: ");
                    String address = scanner.nextLine();
                    System.out.println("Enter Number of Floors: ");
                    int numberOfFloors = scanner.nextInt();

                    Building building = new Building(name, address,
numberOfFloors);

                    System.out.println("Building added successfully!");
                    break;
                case 2:
                    System.out.println("Enter Hospital Name: ");
                    String hospitalName = scanner.nextLine();
                    System.out.println("Enter Hospital Address: ");
                    String hospitalAddress = scanner.nextLine();
                    System.out.println("Enter Number of Floors: ");
                    int hospitalFloors = scanner.nextInt();

```



```
        System.out.println("Enter Number of Beds: ");
        int numberOfBeds = scanner.nextInt();

        Hospital hospital = new Hospital(hospitalName,
hospitalAddress, hospitalFloors, numberOfBeds);

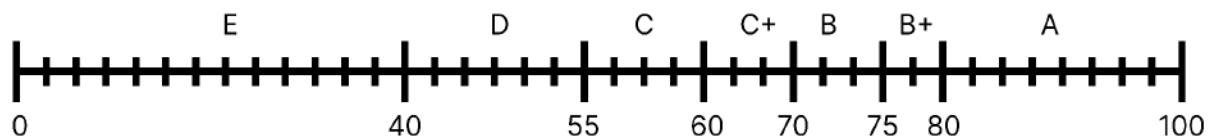
        break;
    case 3:
        System.out.println("Exiting the system. Goodbye!");
        break;
    default:
        System.out.println("Invalid option. Please try
again.");
    }
} while (option != 3);
}
```



Assessment

ASSESSMENT ASPECT	POINT
Exercise	0%
Assignment	100%
Code Running Smoothly	10%
Code Creativity	10%
Answer Questions About the Module	80%
Total	100%

Percentage Assessment



- A = (81 - 100) → You're The Chosen One
- B+ = (75 - 80) → Great!
- B = (70 - 74) → Good!
- 67 = (67) → Square root of 4489
- C+ = (60 - 69) → Keep it up!
- C = (55 - 59) → Almost
- D = (41 - 54) → Study Some More!
- E = (0 - 40) →



Closing Statement?

How is the poem's last module? Was it cool? But it's not as cool as passing this module! Here's the thing: you must pass this module, if not you'll "ketinggalan kereta" because the content is continuous. And yeah, here's a picture of a cat... of..... cat.

